ProjectMatch – AI-Powered Student Project & Team Formation Platform

ProjectMatch helps students find the right teammates for college projects, hackathons, internships and technical competitions. Students build a profile (skills, interests, experience, preferred tech, availability); the platform recommends teammates with **complementary** skills, and then gives the team everything it needs to work together: invitations, real-time chat, task boards, notifications, ratings and an admin dashboard.

Industry: EdTech / Student Collaboration
Type: Full-stack web application + AI recommendation system

Features (Core Modules)

#	Module	Summary
1	Authentication & Profiles	Register/login with JWT (access + refresh), email verification, profile editing
2	Skill & Interest Management	Curated skill taxonomy, proficiency levels, interests, availability
3	AI Team Matching	Hybrid recommender (embeddings + rule-based scoring) with LLM-generated explanations
4	Project / Hackathon Listings	Create, browse, filter and search opportunities with required roles/skills
5	Team Formation	Apply / invite / accept / decline flows, role slots, team lifecycle
6	Real-Time Chat	Team and direct chat via Socket.IO, typing indicators, read receipts
7	Notifications	In-app (real-time) + email notifications for invites, messages, tasks
8	Task Management	Kanban board per team: assignees, due dates, status, comments

9	Ratings & Feedback	Peer reviews after project completion, feeding reputation score
10	Admin Dashboard	User/project moderation, reports, analytics, skill taxonomy management

---

Tech Stack

Layer	Technology
Frontend	React.js (Vite), React Router, Redux Toolkit / Zustand, Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB (Mongoose) – PostgreSQL is a viable alternative, see `architecture.md`
Auth	JWT (short-lived access token + refresh token in httpOnly cookie)
Real-time	Socket.IO
AI	Text embeddings + cosine similarity, rule-based complementarity scoring, LLM for explanations
Cache / Queue	Redis (optional but recommended): Socket.IO adapter, rate limiting, job queue (BullMQ)
Deployment	Vercel (frontend) + Render or AWS (backend, DB on MongoDB Atlas)

Quick Start

Prerequisites

- Node.js 20+
- MongoDB (local or Atlas URI)
- (Optional) Redis
- An LLM / embeddings API key

1. Clone and install

```
git clone https://github.com/<your-org>/projectmatch.git
cd projectmatch
cd server && npm install
cd ../client && npm install
```

2. Configure environment

`server/.env`

```
NODE_ENV=development
PORT=5000
CLIENT_URL=http://localhost:5173
MONGO_URI=mongodb://localhost:27017/projectmatch
JWT_ACCESS_SECRET=change_me
JWT_REFRESH_SECRET=change_me_too
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d
REDIS_URL=redis://localhost:6379
LLM_API_KEY=your_key
EMBEDDING_MODEL=your_embedding_model
LLM_MODEL=your_llm_model
SMTP_HOST=smtp.example.com
SMTP_USER=user
SMTP_PASS=pass
```

`client/.env`

```
VITE_API_URL=http://localhost:5000/api/v1
VITE_SOCKET_URL=http://localhost:5000
```

3. Seed and run

```
cd server && npm run seed   # skills taxonomy + demo users
npm run dev                 # API on :5000
cd ../client && npm run dev # UI on :5173
```

Documentation

File	Purpose
`README.md`	Overview, features, setup (this file)
`architecture.md`	System design, data model, AI pipeline, real-time design, security, deployment
`structure.md`	Folder and file layout for client and server
`implementation.md`	Phase-by-phase build plan with code samples, API list, testing

---

Scripts

Command	Where	Description
`npm run dev`	server / client	Start in development mode
`npm run build`	client	Production build
`npm start`	server	Start production server
`npm run seed`	server	Seed skills and demo data
`npm test`	both	Run test suites
`npm run lint`	both	Lint code

Roadmap

- [x] MVP: auth, profiles, listings, teams
- [ ] AI matching + explanations
- [ ] Chat, notifications, tasks
- [ ] Ratings and admin dashboard
- [ ] Future: GitHub/LinkedIn import, calendar sync, mobile app, college-level leaderboards

---

Contributing

1. Fork the repo and create a feature branch (`feat/<name>`)
2. Follow ESLint/Prettier rules; write tests for new logic
3. Open a PR with a clear description

License

MIT